tierlm 0.2.0

A local LLM engine that runs models bigger than your graphics card, by spreading them across VRAM, RAM and (soon) SSD

Version	0.2.0 (October 27, 2026)
What it is	A from-scratch C++ inference engine with a web chat and live dashboard. Windows and Linux; Intel, AMD and NVIDIA GPUs.
Built by	Written by Claude (Anthropic's AI), with Chad Jameson (EHTi) directing the project, making the design calls, and testing it on real hardware.
Licence	MIT. Copyright (c) 2026 Chad Jameson (EHTi).
Source	github.com/EHTi-Chad/tierlm

Contents

1. What tierlm is · 2. What's new in 0.2.0 · 3. Supported models · 4. Getting started · 5. The settings · 6. How it works · 7. Test results · 8. Troubleshooting · 9. Building from source · 10. Known limits and what's next

1. What tierlm is

Large language models are usually run on servers with hundreds of gigabytes of graphics memory. At home, the graphics card (GPU) is the fast part but has the least memory: 8 GB on a typical laptop card, 32 GB on the biggest consumer card. System RAM is bigger but several times slower, and an SSD is bigger again and slower still.

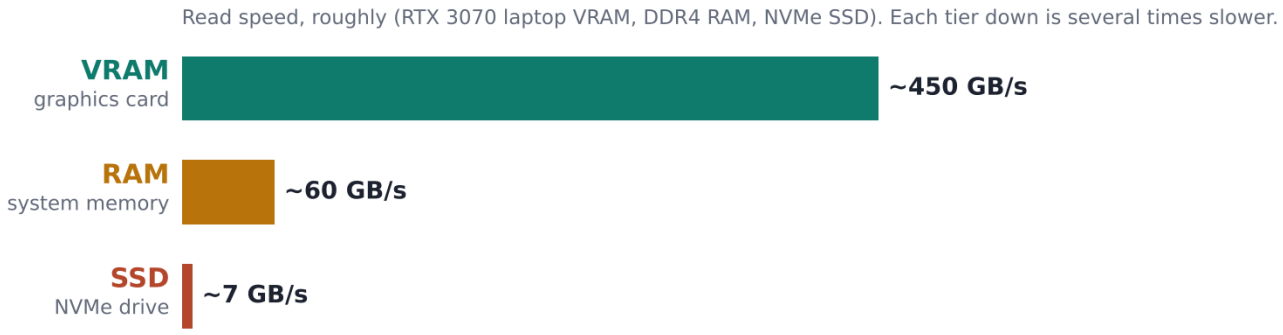


Figure 1. The three memory tiers. tierlm puts the parts of a model that are used most in the fastest tier that has room.

tierlm loads a model in the common GGUF format, works out where every piece of it should live, and runs it. If the model fits on the GPU, it runs there. If it doesn't, tierlm splits it: as much as fits goes into VRAM and the rest into RAM, where the CPU works on it. The idea comes from **Strata**, an open-source engine that runs a 125-billion-parameter model on a 12 GB GPU. tierlm is its own engine, built on the same principle:

- **Fast**: C++, with the ggml library's math kernels for CPU, Vulkan and CUDA.
- **Reads common models**: the Llama, Mistral, Qwen and OLMoE families, straight from the files people download from Hugging Face or already have in LM Studio.

- **Tiered memory**: VRAM first, then RAM, then SSD, decided automatically.
- **Nothing to install**: unzip, double-click, and pick a model in the browser. The only requirement is a normal graphics driver.

tierlm does **not** use llama.cpp. The only outside engine code is ggml, a small MIT-licensed library of fast matrix kernels and the quantized file formats. The tokenizers, the model graphs, the memory planning, the tier loader, the server and the web app are all tierlm's own.

2. What's new in 0.2.0

Version 0.1 ran one model family (OLMoE) on one kind of GPU, with a fixed setup. 0.2.0 adds:

Area	New in 0.2.0
Models	Seven model variants instead of one: Llama 2 and 3.x, Mistral 7B, Mixtral, Qwen2/2.5, Qwen3, Qwen3 MoE and OLMoE. Three tokenizer styles (Llama 3, Qwen, SentencePiece) plus GPT-2. The chat format is read from each model file. Big models split over several files (-00001-of-0000N.gguf) load as one model.
Tier loader	Models bigger than VRAM are split between VRAM and RAM automatically, by whole layers (default) or with experts in RAM. A VRAM limit setting lets you reserve VRAM or simulate a smaller card.
NVIDIA	CUDA support, tested on an RTX 3070 laptop GPU. Vulkan also works on NVIDIA, and was faster than CUDA for models that fit in VRAM.
Setup screen	Hardware is detected and used automatically (GPU first, then CPU). A models folder with a Windows folder picker, subfolder search (LM Studio's layout) and a one-click LM Studio folder option. Context length up to each model's own maximum, with the memory each size needs.
Settings	LM Studio-style options, remembered per model: flash attention, KV cache precision, prompt batch size, active experts, seed, split mode, VRAM limit, Force CPU. Sampling: repetition penalty, top-k, top-p, min-p.
Reliability	A clear warning (with the fix) when leftover VK_ICD_FILENAMES / VK_DRIVER_FILES settings hide the GPU. Vision add-on (mmproj) files are labelled as such. Version shown in the app.
Testing	Every architecture checked against an independent reference, also when split between a GPU and RAM (140 checks). Tokenizers checked against Hugging Face on real models.

3. Supported models

tierlm reads the architecture from the file. The setup screen marks every model as supported or says why not.

Family	GGUF architecture	Examples	Status
OLMoE	olmoe	OLMoE-1B-7B-0924-Instruct	Tested on real hardware
Qwen3	qwen3	Qwen3-0.6B to Qwen3-32B	Tested: Qwen3-0.6B
Qwen3 MoE	qwen3moe	Qwen3-30B-A3B, Qwen3-235B-A22B	Tested: Qwen3-30B-A3B
Llama 3 / 3.1 / 3.2 / 3.3	llama	Llama-3.2-1B, Llama-3.1-8B, Llama-3.3-70B	Tested: Llama-3.2-1B
Llama 2, Mistral 7B, TinyLlama	llama (SentencePiece)	Mistral-7B-Instruct, TinyLlama-1.1B-Chat	Tested: TinyLlama-1.1B

Family	GGUF architecture	Examples	Status
Mixtral	llama (with experts)	Mixtral-8x7B	Math verified; not yet run on real weights
Qwen2 / 2.5	qwen2	Qwen2.5-7B-Instruct, Qwen2.5-Coder	Math verified; not yet run on real weights
Qwen3.5 hybrid, Gemma, Phi, DeepSeek, gpt-oss, ...	various	Qwen3.8-27B, Gemma 3	Not yet

Models are often quantized by different people (bartowski, Unsloth, the model makers themselves); all of them work as long as the family is supported, including Unsloth's mixed "dynamic" quants. Big models that come as several files (`-00001-of-00003.gguf` ...) load as one model: put all the parts in the same folder. A file called `mmproj-*.gguf` next to a model is its **vision add-on** (for reading images); tierlm lists it, but doesn't use images yet.

4. Getting started

This is everything a new user needs. No programming, no Python, no CUDA toolkit, no SDKs.

What you need

Item	Details
Windows 10 or 11, 64-bit	Any CPU with AVX2 (almost every Intel or AMD CPU since 2015). Linux works too, built from source.
A graphics driver	Vulkan support comes with every current NVIDIA, AMD and Intel driver. Keep it up to date. Without a GPU, tierlm runs on the CPU.
The tierlm zip	About 20-30 MB, from the Releases page.
Memory for the model	The model's file size, plus a little for the conversation. It can be spread over VRAM and RAM.
A model file (.gguf)	From Hugging Face, or the ones you already have in LM Studio.

Step by step

1. **Download** `tierlm-0.2.0-windows-x64-vulkan.zip` from github.com/EHTi-Chad/tierlm/releases. Right-click it, choose **Extract All...**, and extract it somewhere simple, such as `C:\tierlm`.
2. **Get a model.** If it comes as several files (`-00001-of-00003.gguf` ...), download all of them. To start, download `OLMoE-1B-7B-0924-Instruct-Q4_K_M.gguf` (4.2 GB) or `Llama-3.2-1B-Instruct-Q4_K_M.gguf` (0.8 GB) from Hugging Face, and put it in the `models` folder inside tierlm. If you already have models elsewhere (LM Studio's folder, for example), leave them there: step 4 points tierlm at them.
3. **Start it.** Double-click `start-tierlm.bat`. If Windows shows *Windows protected your PC*, click **More info**, then **Run anyway** (tierlm isn't signed with a paid certificate yet). A black window opens: leave it open while you use tierlm. Your browser opens `http://127.0.0.1:8040`.
4. **Pick a model** on the setup screen. **Detected hardware** shows your GPU and CPU and which one will be used, with a bar showing where the model will go (VRAM, RAM, SSD). **GGUF models** lists the models in your folder; the **⋯** menu lets you **Choose models folder...** (a normal Windows folder picker), use LM Studio's folder, include subfolders, or open Hugging Face.
5. **Choose settings** (or keep the defaults) and click **Load and start chatting**. Loading takes from a few seconds to a minute for big models. The first time on a GPU, the driver also prepares its programs once.
6. **Chat.** The right side is the live dashboard: speed, how full the conversation memory is, where the model lives, and, for mixture-of-experts models, which experts are working. **Change setup** (top right) brings the setup screen back.
7. **Stop** by closing the black window.

Tip: tierlm remembers your models folder and, like LM Studio, your settings for each model, in `tierlm-settings.json`. Nothing leaves your computer: the page only talks to 127.0.0.1.

5. The settings

Setting	What it does	Default
Context length	How much conversation the model remembers. Options go up to the model's own maximum, with the memory each size needs.	4K tokens
CPU threads	Threads for work on the CPU. On laptops, fewer can be faster.	half your threads
Flash attention	Faster attention that reads the conversation memory once. Off only to compare or troubleshoot.	on
KV cache precision	f16 uses half the memory; f32 is exact but twice the size.	f16
Prompt batch size	Tokens read at once when processing a prompt. Bigger is faster for long prompts, but uses more memory.	256
Active experts	Mixture-of-experts models only: experts used per layer per token. Fewer is faster, lower quality.	the model's own
Seed	-1 is random. Any other number makes replies repeatable.	-1
When it doesn't fit	Split by whole layers (one GPU/CPU hand-off per token), or attention in VRAM with experts in RAM.	whole layers
VRAM limit	Use at most this much VRAM. Keeps room for other apps, or simulates a smaller card.	0 (all free)
Force CPU	Ignore the GPU. Useful for troubleshooting.	off
Temperature	In the chat panel. 0 always picks the most likely word; higher is more varied.	0.70
Repeat penalty	In the chat panel. Makes recently used words less likely, which stops loops.	1.10
Top-k / Top-p / Min-p	Which candidate words are considered. Same meaning and defaults as LM Studio.	40 / 0.90 / 0.05

6. How it works



Figure 2. How one word is produced. tierlm repeats this for every word of a reply.

Each model family is a variation on the same transformer design. tierlm describes the differences with a handful of switches (how positions are encoded, whether attention is normalised, whether the feed-forward part is one network or many experts, how expert weights are combined), so one graph builder runs all of them. The tokenizer and chat format are read from the model file.

Mixture-of-experts (MoE) models

In an MoE model, each layer has many small "expert" networks (128 in Qwen3-30B-A3B) and a router picks a few of them (8) for each word. So a 30-billion-parameter model only does about 3 billion parameters of work per word. The experts are most of the file, but each one is rarely needed. That makes MoE models the best case for tiering: the parts used for every word go in fast memory, and the experts can wait in slower memory.

The tier loader

When a model doesn't fit in VRAM, tierlm decides where every tensor goes before loading. In the default **whole layers** mode, it fills VRAM from the last layer backwards (each layer with its share of the conversation memory, the KV cache), plus the output layer. The first layers stay in RAM and run on the CPU. Each word therefore crosses from CPU to GPU once.

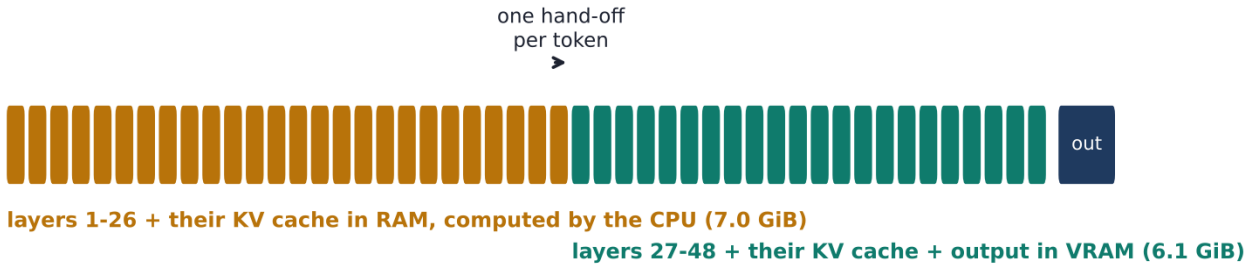


Figure 3. How Qwen3-30B-A3B (14.7 GB) was placed on an 8 GB RTX 3070: 22 of 48 layers in VRAM.

Finding the hidden hand-offs

The first tiered version was slower than running on the CPU alone. A profiler built for this showed why: ggml's scheduler sends any step without weights of its own (normalisation, attention over the conversation memory) to the GPU if it can, even in the middle of a layer that lives in RAM. That meant 80 to 137 trips between CPU and GPU per word, and on Windows each trip waits for the GPU. tierlm now pins every step of a RAM layer to the CPU, and the trips dropped to 2.

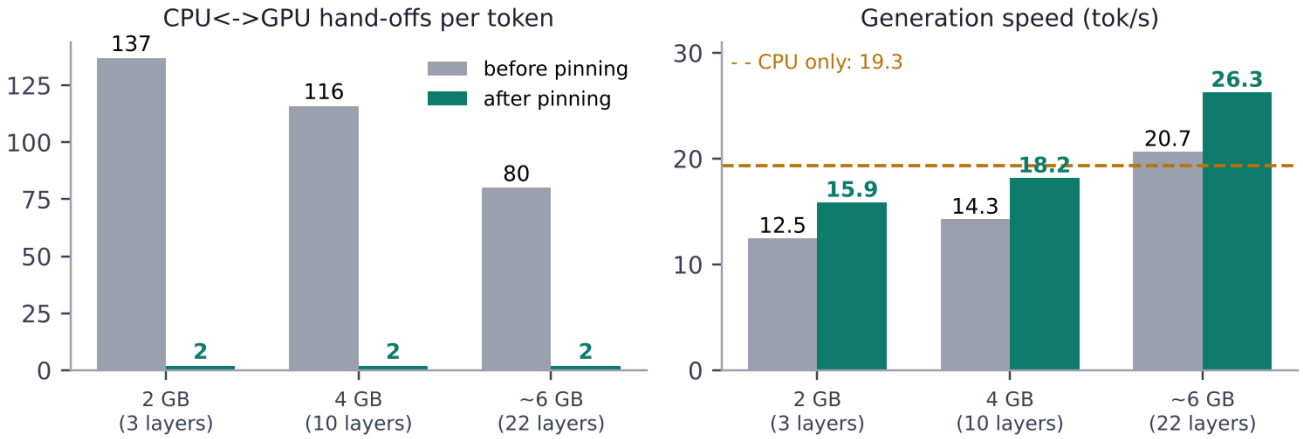


Figure 4. Qwen3-30B-A3B on the RTX 3070 at three VRAM sizes, before and after pinning RAM layers to the CPU.

7. Test results

Correctness first

Speed only counts if the answers are right. Every change was checked against references that don't share any code with tierlm:

Check	How	Result
Model math, every architecture	Tiny random models of each family, compared with a plain NumPy implementation written from the architecture's definition. Exact (f32) and normal (f16) conversation memory, one-shot and word-by-word, prompts over 256 tokens.	All 7 variants match to about 2e-6
Splitting between GPU and RAM	The same checks with the model split at 5 VRAM sizes in both split modes, using a stand-in GPU (ggml's RPC backend).	140 of 140 pass
Models in several files	Every family written as a 3-part split GGUF by Hugging Face's gguf library, loaded through part 1.	All 7 match the reference
Tokenizers (logic)	Small tokenizers trained with Hugging Face's tokenizers and Google's sentencepiece libraries, 31 tricky strings each (capitals, numbers, emoji, Cyrillic, Japanese, Windows line endings).	All exact, incl. both SentencePiece storage formats
Tokenizers (real models)	29 strings against each model's official Hugging Face tokenizer.	Qwen3-0.6B, Llama-3.2-1B and TinyLlama: 29/29 each
Real weights	"The capital of France is" on real models, top prediction.	" Paris": Qwen3-0.6B 65%, Llama-3.2-1B 68%, TinyLlama 37%, Qwen3-30B-A3B 69-73% (CPU, CUDA, split)

Intel Arc 140V laptop (Vulkan)

Intel Core Ultra 7 258V with Arc 140V integrated graphics, 32 GB shared memory, Windows 11. Model: OLMoE-1B-7B-Instruct Q4_K_M (3.9 GB). Speed falls as the conversation grows, because each new word looks back over all earlier ones; the f16 conversation memory and flash attention made long conversations much faster.

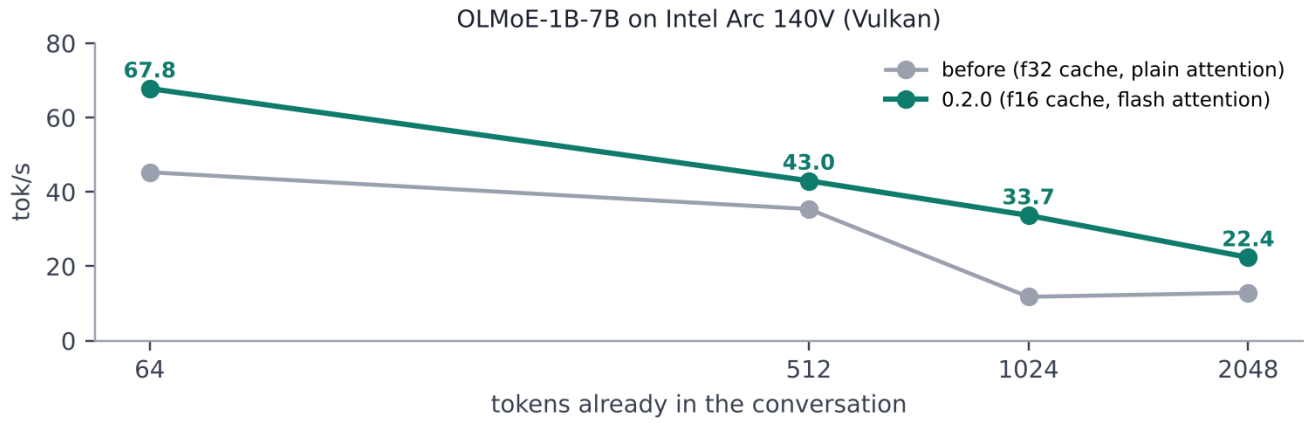


Figure 5. Generation speed against conversation length on the Arc 140V. For comparison, the same laptop's CPU (under WSL) ran it at about 46 tok/s at short context.

RTX 3070 laptop (CUDA and Vulkan)

AMD Ryzen 9 5900HX, NVIDIA GeForce RTX 3070 Laptop GPU (8 GB), 32 GB RAM, Windows 11, CUDA 13.4. Generation speed over 100-200 words.

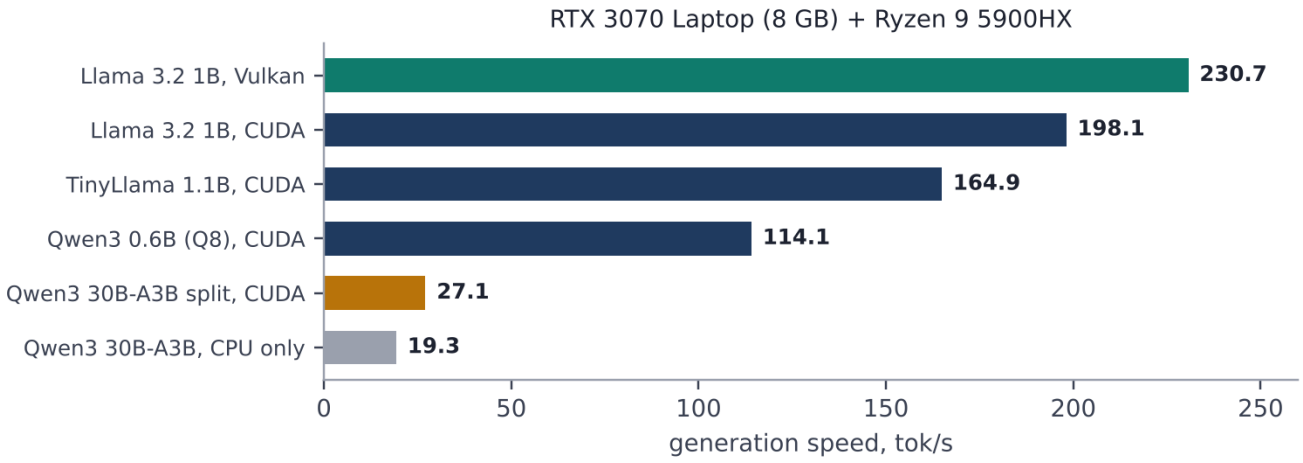


Figure 6. Small models run entirely in VRAM; Qwen3-30B-A3B (14.7 GB) is split between VRAM and RAM.

Qwen3-30B-A3B Q3_K_M, 14.7 GB	Placement	Generation	Prompt
CPU only	all in RAM	19.3 tok/s	32 tok/s
CUDA, VRAM limited to 2 GB	3 of 48 layers in VRAM	15.9 tok/s	24 tok/s
CUDA, VRAM limited to 4 GB	10 of 48 layers in VRAM	18.2 tok/s	25 tok/s
CUDA, all free VRAM	22 of 48 layers (6.1 GB) in VRAM	26.3 tok/s	33 tok/s
CUDA, all free VRAM, 6 CPU threads	22 of 48 layers in VRAM	27.1 tok/s	31 tok/s
Vulkan, all free VRAM	22 of 48 layers in VRAM	16.1 tok/s (first run, driver still preparing; to be re-measured)	2 tok/s

What the numbers say. With 22 of 48 layers on the GPU, the 30-billion-parameter model runs about 40% faster than on the CPU alone. On this laptop the CPU part runs about 25% slower whenever the GPU is busy. CUDA loaded with no GPU work was not slower, which points to the CPU and GPU sharing the laptop's power budget, so desktops should gain more. For a model that fits in VRAM, Vulkan was 16% faster than CUDA once its programs were prepared (230.7 against 198.1 tok/s).

8. Troubleshooting

Problem	What to do
"Windows protected your PC"	Click More info, then Run anyway. You can check the zip against the .sha256 file on the Releases page first.
The GPU isn't listed, or a yellow box mentions VK_ICD_FILENAMES / VK_DRIVER_FILES	Leftover settings from an old driver install point Vulkan at a driver file that no longer exists. Run the reg delete command shown in the yellow box, then sign out of Windows and back in. Tell-tale sign: Vulkan works as administrator but not as your normal account.
The GPU still isn't listed	Update the graphics driver. Run tierlm-devices.exe in the tierlm folder: it lists what tierlm can see and why a GPU failed.
The page shows a different app, or "404"	Another program uses the port. Start tierlm on another one: start-tierlm.bat 8050, then open http://127.0.0.1:8050. Use 127.0.0.1, not localhost.
The page still shows an old version	An older tierlm is still running in another window and owns the port. Close every tierlm window and start again.
"No models found"	Put a .gguf file in the models folder, or use ⋯ → Choose models folder..., then Rescan.
A model is "not supported yet"	The label says which architecture or tokenizer the file uses. See section 3. A file labelled Vision add-on (mmproj) is not a model on its own.
Replies repeat or ramble	Raise Repeat penalty a little (e.g. 1.15). Very small models loop more easily, and can state wrong things confidently.
It's slow	Check the Where the model lives panel: a model split with RAM is slower than all-VRAM. Try a smaller quantization, a shorter context, or fewer CPU threads on laptops.
"not enough memory"	Pick a shorter context, close other programs, or pick a smaller model. The SSD tier isn't built yet, so model + context must fit in VRAM + RAM.

9. Building from source

Only needed to change tierlm or build a release. On Windows: Visual Studio 2022 or 2026 with Desktop development with C++, the Vulkan SDK, and for CUDA the CUDA Toolkit. Then, in a Developer Command Prompt:

```
git clone --recurse-submodules https://github.com/EHTi-Chad/tierlm.git
cd tierlm
cmake -S . -B build -G Ninja -DTIERLM_VULKAN=ON -DTIERLM_CUDA=ON
cmake --build build
build\tierlm-devices.exe
```

`scripts\package-windows.bat 0.2.0` builds the portable release zip (AVX2 baseline, C runtime included). docs/BUILD.md covers Linux, WSL, every test, and the pitfalls hit during development (an accidental Debug build, stale Vulkan settings, ports in use, the CUDA compiler flag).

10. Known limits and what's next

- **SSD tier**: not built yet. Model and context must fit in VRAM plus RAM.
- **Hot-expert cache**: fill VRAM with the experts the router picks most, instead of whole layers. The dashboard already measures how much this would help.

- **More families**: the Qwen3.5/3.6 hybrid (linear attention), Gemma, Phi, DeepSeek, gpt-oss; and real-weight tests of Mixtral and Qwen2.5.
- **CUDA for everyone**: the release zip is Vulkan-only (small, works on all cards). A one-click CUDA download for NVIDIA cards is planned if CUDA proves faster for split models.
- **Qwen3 thinking** is shown as raw <think> text for now.
- Quantized KV cache, OpenAI-compatible API, a Hugging Face Space, AMD testing.

tierlm is MIT-licensed. ggml, cpp-httplib and nlohmann/json keep their own MIT licences (THIRD_PARTY_NOTICES.md). Model weights are not included and have their own licences.